

LLM Slide:

Reasoning models are not fundamentally different architectures — they are *trained and optimized differently to perform multi-step reasoning explicitly*.

So:

- ❌ Not a completely new model type
- ✅ Same base (Transformer)
- ✅ Different **training + inference behavior**



1. Base Similarity (Very Important)

Both:

- “Normal LLMs” (like OpenAI GPT models)
- “Reasoning models” (like OpenAI o1 / DeepSeek R1)



Use the same core:

Transformer architecture

So the difference is NOT:

- Not attention
- Not embeddings
- Not architecture



2. Where the REAL Difference Lies



A. Training Objective



Normal LLM

Optimized for:

Next-token prediction

Mathematically:

$$P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

$$P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

Goal:

- Predict the next word as accurately as possible



Reasoning Model

Optimized for:

Multi-step correctness (not just next token)

They are trained to:

- Generate **intermediate reasoning steps**
- Arrive at **correct final answers**

◆ B. Chain-of-Thought (CoT) Behavior

Normal LLM:

- Can reason if prompted
- But reasoning is:
 - ❌ implicit
 - ❌ shallow
 - ❌ inconsistent

Reasoning Model:

- Trained to:
 - ✅ break problems into steps
 - ✅ verify intermediate steps
 - ✅ self-correct

Example:

Normal LLM:

Q: 17×24 ?

A: 408

Reasoning Model:

$$\begin{aligned} 17 \times 24 &= (17 \times 20) + (17 \times 4) \\ &= 340 + 68 \\ &= 408 \end{aligned}$$

◆ C. Test-Time Compute (Huge Difference)

This is the **biggest practical difference**.

Normal LLM:

- One-shot generation
- Fast
- No thinking loop

Reasoning Model:

- Uses **deliberate thinking**
- May:
 - Generate multiple paths

- Evaluate them
- Pick best answer

👉 This is often called:

"Inference-time scaling"

◆ D. Training Techniques Used

Reasoning models often include:

1. Chain-of-Thought Fine-Tuning

- Trained on step-by-step solutions

2. Reinforcement Learning (RL)

- Reward:
 - Correct answers
 - Logical steps

3. Self-Consistency

- Generate multiple answers
- Choose most consistent one

4. Tool Use / Scratchpads

- Intermediate memory
- Structured reasoning traces

🧩 3. Intuition (Best Way to Explain to Audience)

🎯 Analogy

👉 Normal LLM:

"A smart student who answers quickly"

👉 Reasoning Model:

"A student who shows all steps, double-checks, and then answers"

🔬 4. Internal Behavior Difference

Normal LLM:

- Compresses reasoning into hidden states
- You don't see the steps

Reasoning Model:

- Externalizes reasoning:
 - Writes steps
 - Uses them to improve accuracy

⚠️ 5. Trade-offs

Feature	Normal LLM	Reasoning Model
Speed	⚡ Fast	🐢 Slower
Cost	💰 Lower	💰 Higher
Accuracy (simple tasks)	✅ Good	✅ Good
Accuracy (complex tasks)	❌ Struggles	✅ Strong
Transparency	❌ Low	✅ High

🧠 6. Deep Technical Insight (Important for You)

Reasoning models effectively:

Transform inference into a search problem

Instead of:

input → output

They do:

input → reasoning paths → evaluation → best output

🧑 7. How YOU Should Explain in Class

Use this exact line:

“Normal LLMs predict the next word. Reasoning models simulate thinking before answering.”

Then draw:

Normal LLM:

Q → Answer

Reasoning Model:

Q → Step1 → Step2 → Step3 → Answer

🚀 Final One-Liner

A reasoning model is an LLM that is trained and optimized to think in steps, not just predict tokens.

Also Talk About Open Source & Close Source Ecosystem:

<https://huggingface.co/MiniMaxAI/MiniMax-M2.7>

<https://huggingface.co/Qwen/Qwen3.6-35B-A3B>

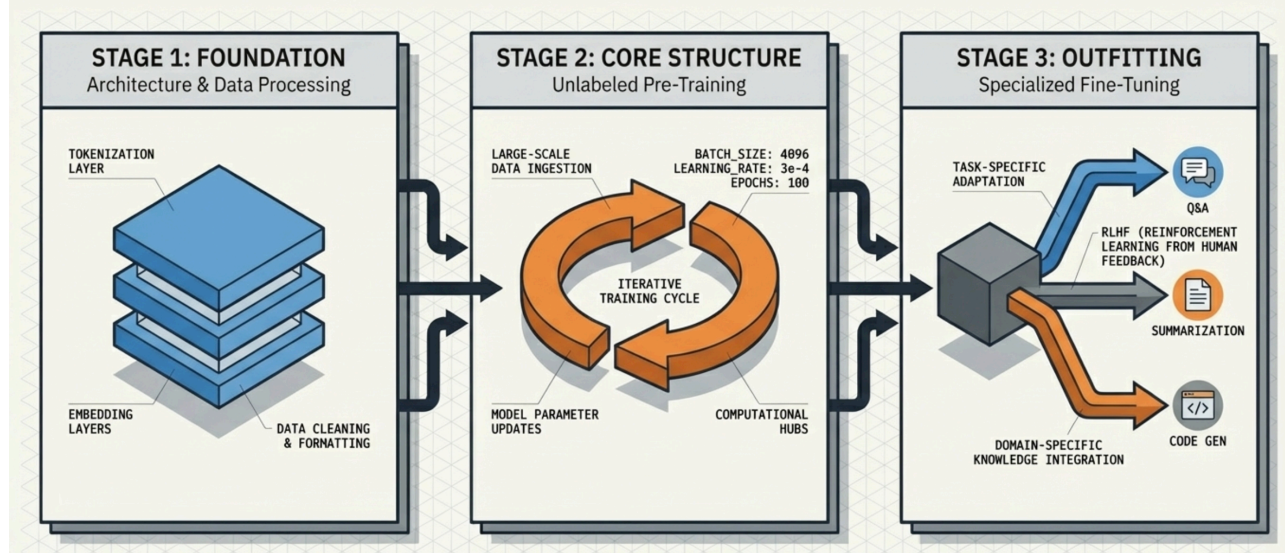
Tokens Slide:

I/p Token , O/p Token Costing for models.

- 1) <https://openai.com/api/pricing/>
2. <https://azure.microsoft.com/en-us/pricing/details/azure-openai/>
3. <https://ai.google.dev/gemini-api/docs/pricing>

Tokenization:

The three-stage blueprint for LLM construction.



Here Tokenization Comes into the picture.

- Normal & Byte Pair Encoding Techniques for Tokenization (Show them notebook), don't implement it.
- Talk about toktoken library: <https://github.com/openai/tiktoken>
- Tokenizer Playground: <https://huggingface.co/spaces/Xenova/the-tokenizer-playground>

Embeddings:

Converting the tokens(token-ids) into meaning full dense vector representation.

Multiple Embedding Models:

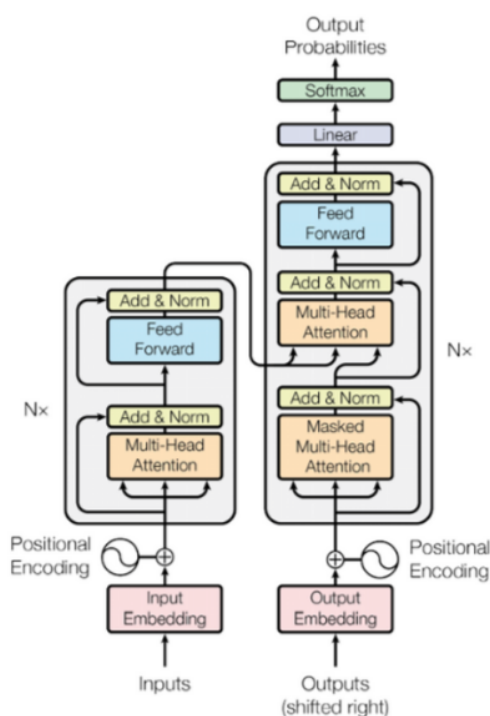
- Text Embedding Models- Text to Vectors : <https://huggingface.co/Xenova/all-MiniLM-L6-v2>
- Image Embedding Models: Image to Vectors : <https://huggingface.co/vidore/colpali-v1.3-hf>
- Multimodal Embedding Models: Multimodal to Vectors : <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-embedding-2/>

***Attention Slide*:** How the Model Reads Context

Birds Eye View On Transformer Architecture:

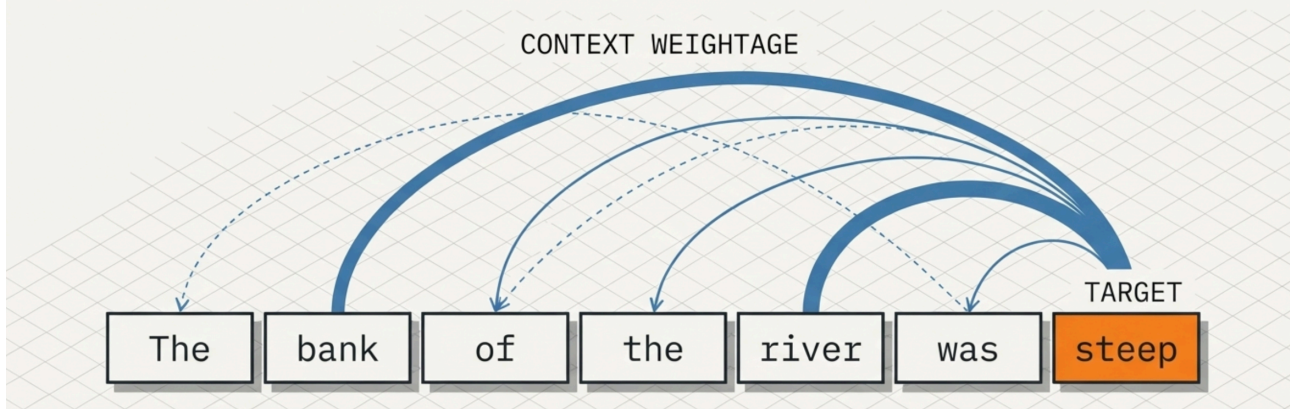
Transformer

Attention Is All You Need

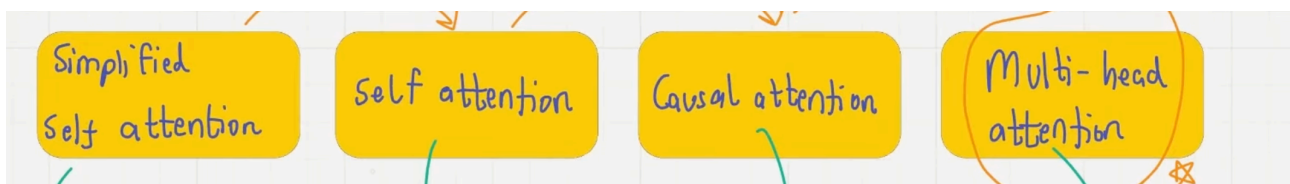


The attention mechanism weights the entire historical context.

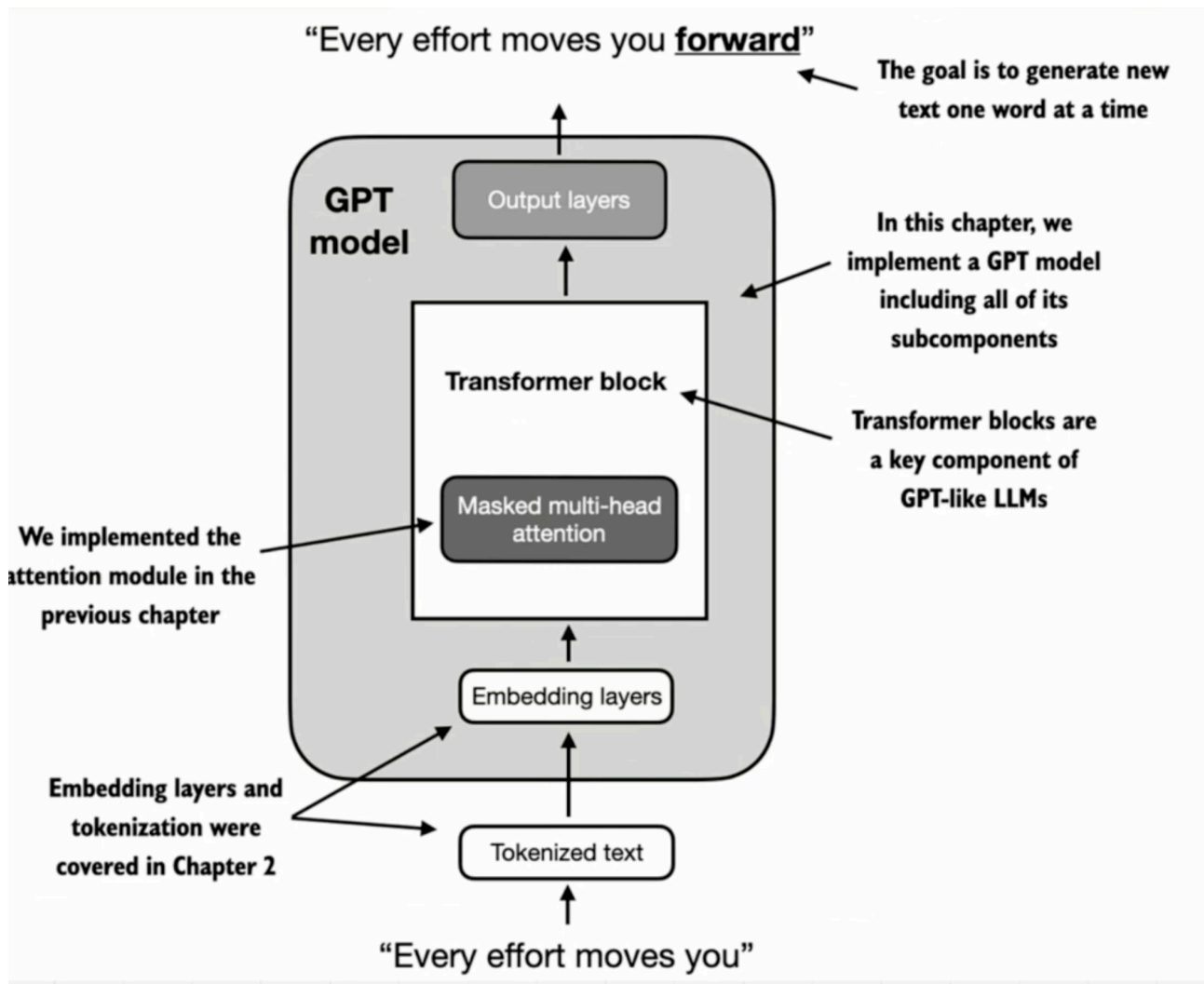
Models do not just look at the immediately preceding word. Attention blocks calculate scores that grant selective access to the entire input sequence, dynamically determining which previous words matter most.



Types of Attention Mechanism:



This is the crux/O/p of Multihead Attention Block:
Context Vectors= (Embedding Vectors) Semantic meaning + How Each token relates to each other tokens (Long Range Dependency)



🧠 GPT-2 Architecture Overview

GPT-2 is a **decoder-only Transformer** with:

- Token + positional embeddings
- N repeated Transformer blocks
- Final normalization + linear output head

High-level flow:

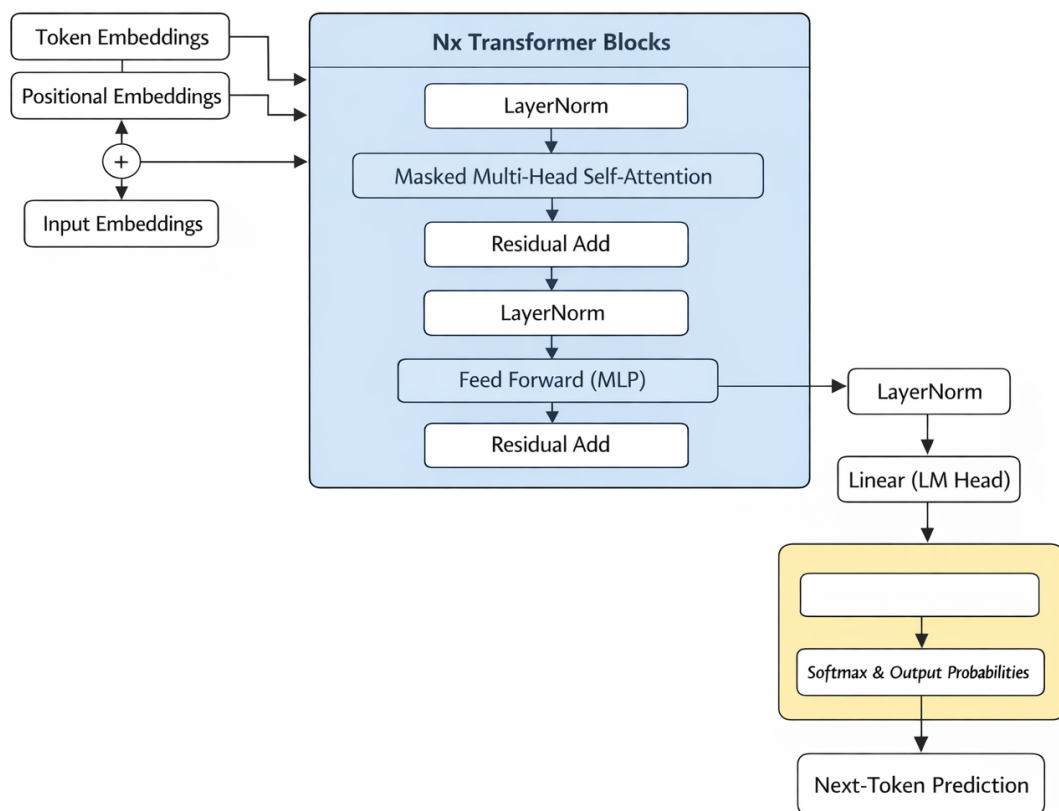
Input tokens



Token Embedding + Positional Embedding



[N × Transformer Block]
↓
LayerNorm
↓
Linear (LM Head)
↓
Softmax → next-token probabilities



1. Input Embedding Layer

Components:

- **Token embeddings:** map token IDs → vectors
- **Positional embeddings:** encode token position

Formula:

$x = \text{token_embedding}(\text{input_ids}) + \text{position_embedding}(\text{position_ids})$

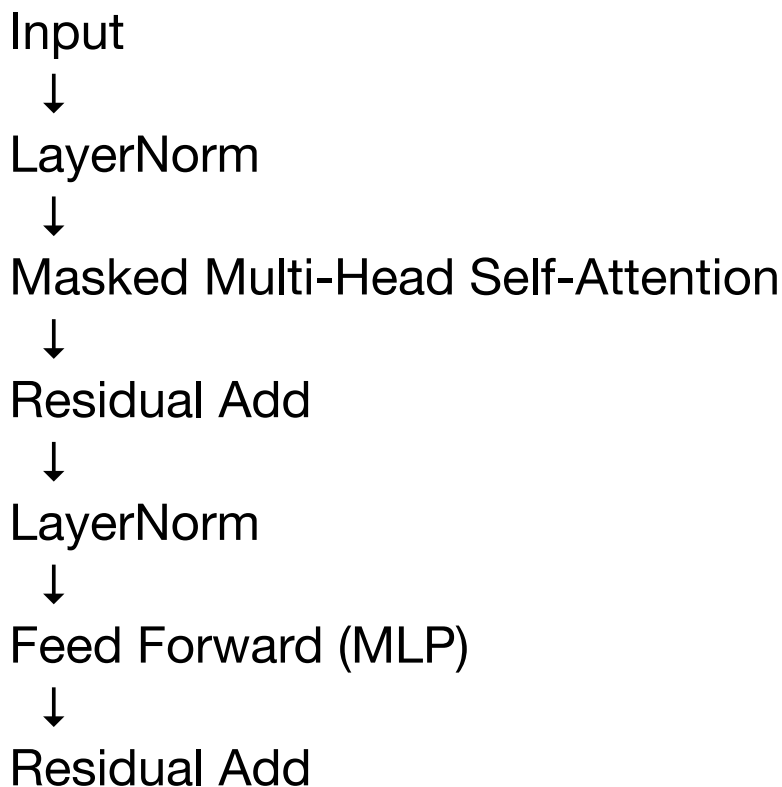
Typical GPT-2 sizes:

- vocab_size = 50,257
- d_model = 768 (small GPT-2)



2. Transformer Block (Core Unit)

Each block contains:



2.1 Masked Multi-Head Self-Attention

Purpose:

Each token attends to **previous tokens only** (causal masking).

Steps:

(1) Linear projections:

$$Q = xW_q$$

$$K = xW_k$$
$$V = xW_v$$

(2) Split into heads:

$$(Q, K, V) \rightarrow (\text{num_heads}, \text{head_dim})$$

(3) Attention scores:

$$\text{scores} = QK^T / \sqrt{d_k}$$

(4) Apply causal mask:

$$\text{Future tokens} = -\infty$$

(5) Softmax:

$$\text{attn} = \text{softmax}(\text{scores})$$

(6) Weighted sum:

$$\text{output} = \text{attn} \cdot V$$

(7) Merge heads + projection:

$$\text{output} = \text{concat}(\text{heads})W_o$$

◆ **2.2 Residual Connection**

$$x = x + \text{attention_output}$$

◆ **2.3 Feed Forward Network (MLP)**

A 2-layer fully connected network:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Typical GPT-2:

- hidden size = 768
- inner FFN = 3072 (4× expansion)

◆ 2.4 Second Residual

$$x = x + \text{FFN_output}$$

3. Stack of Blocks

GPT-2 stacks these blocks:

Model	Layers
GPT-2 small	12
GPT-2 medium	24
GPT-2 large	36
GPT-2 XL	48



4. Final Layers

After all blocks:

LayerNorm:

$$x = \text{LayerNorm}(x)$$

Output projection (LM head):

$$\text{logits} = x \cdot W_{\text{vocab}}^T$$

Often tied weights with embedding matrix.



Key Design Details (Important)

1. Causal Mask

Prevents “seeing the future”:

```
mask = torch.tril(torch.ones(T, T))
```

2. No Encoder

Unlike original Transformer:

- GPT-2 = **decoder only**
- No cross-attention



Mental Model

Think of GPT-2 as:

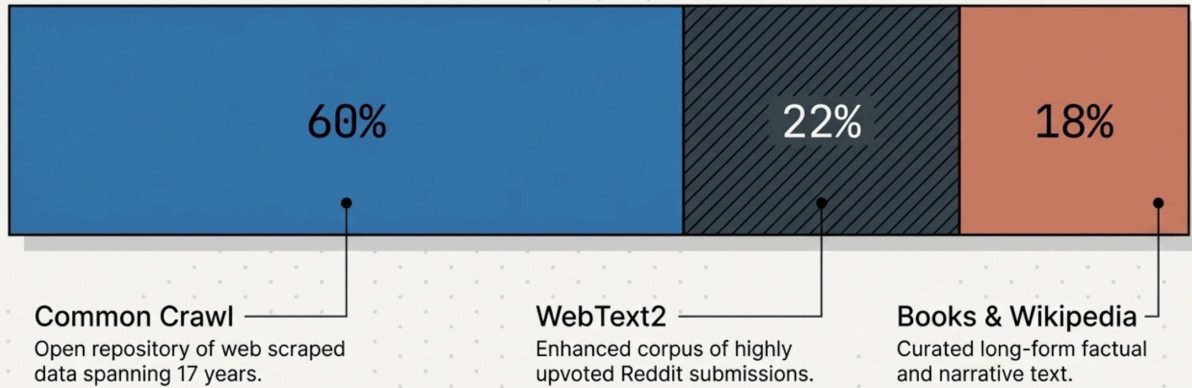
“A deep stack of attention + MLP blocks that repeatedly refine token representations while only looking left.”

Note: We have model like Google T5 Which uses both Encoder & Decoder Block from Transformers Architecture:

Pre training Slide:

Ingesting the Internet: 300 Billion Tokens

A token is roughly equivalent to a single word. To optimize 175 billion parameters, GPT-3 was trained on 300 billion tokens. The **sheer volume** of this text corpus ensures the model maps the statistical relationships of nearly every concept documented by humanity.



<https://commoncrawl.org/>

<https://openwebtext2.readthedocs.io/en/latest/>

Finetuning: : To align the Next word prediction with Human like response (not a single word prediction.)

<https://huggingface.co/sachin19566/bge-base-en-v1.5-course-recommender-v1>

<https://huggingface.co/sachin19566/distilbert-Pizza Intent>

***Supervised FT & RLHF Slide*: (Polite & Helpful)**

RLAIF - Reinforcement learning by AI(Artificial Intelligence) Feedback

Context Window Slide

**Andrej Karpathy Context Engineering
Tweet:**

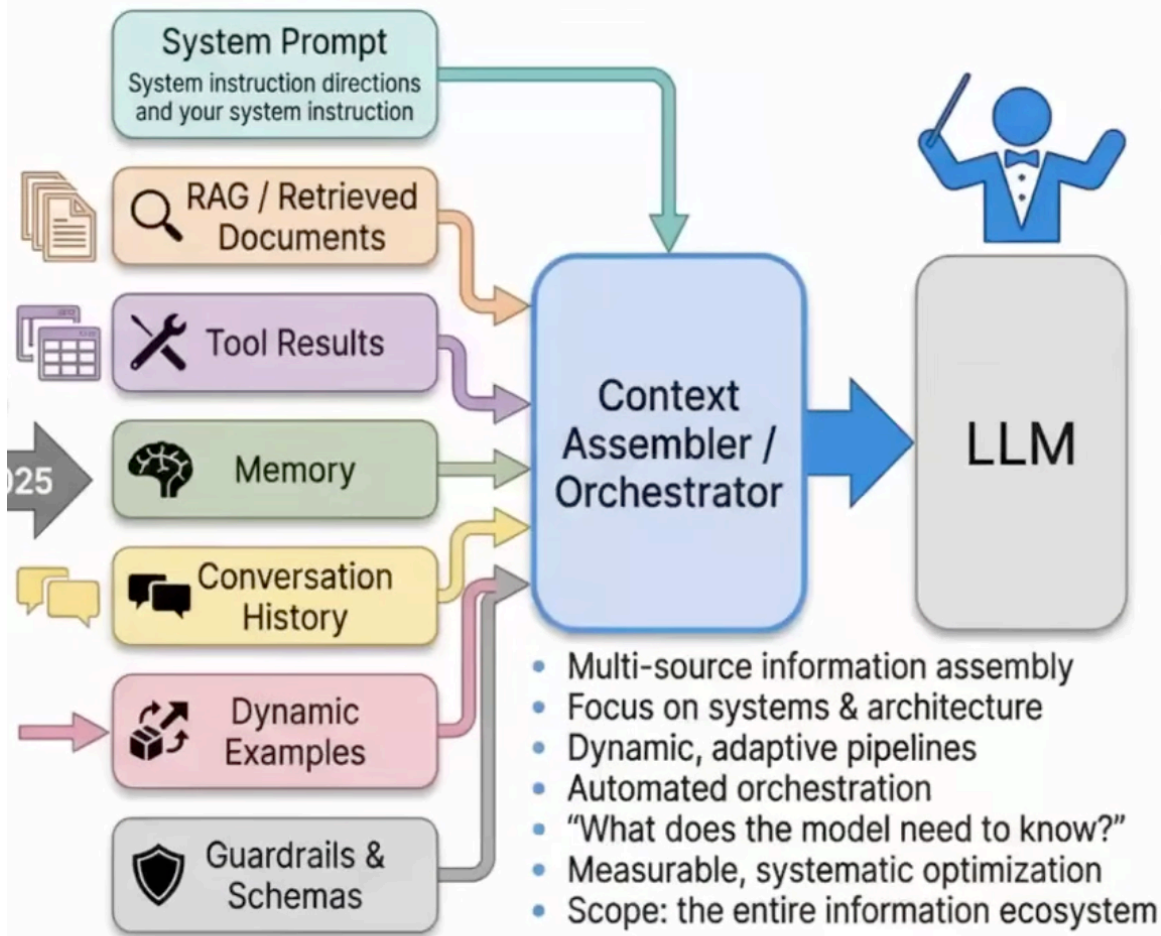
**[https://x.com/karpathy/status/
1937902205765607626?lang=en](https://x.com/karpathy/status/1937902205765607626?lang=en)**

**Openclaw : Manages the Context, It
has soul.md file**

**Similarly : Claude Desktop (leaked
code), manages the Context Very
greatly.**

Context Engineering Elements

CONTEXT ENGINEERING



So It's like a land & you have to fill it properly
(It's limited) Analogy